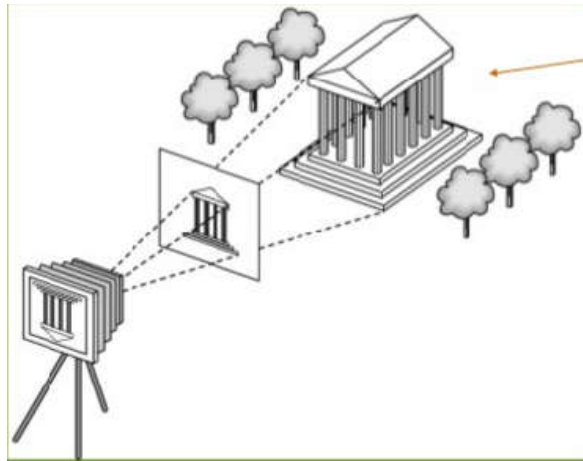


Modelo de representación

Transformaciones Geométricas

2D-3D.



Temario

Tema 1: Introducción: Revisión histórica da computación gráfica

Tema 2: Estándares gráficos Evolución histórica, OpenGL vs DirectX

Tema 3: Formas 2D e Antialiasing (final del curso)

Tema 4: Transformaciones geométricas, 2D, 3D,

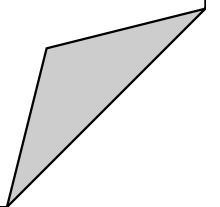
Tema 5: Proyección, modelo de cámara sintética.

Tema 6: Modelado e texturas

Tema 7: Color, Iluminación y sombreado

Tema 8: Determinación de superficies visibles, z-Buffer

Tema 9: Shaders GLSL



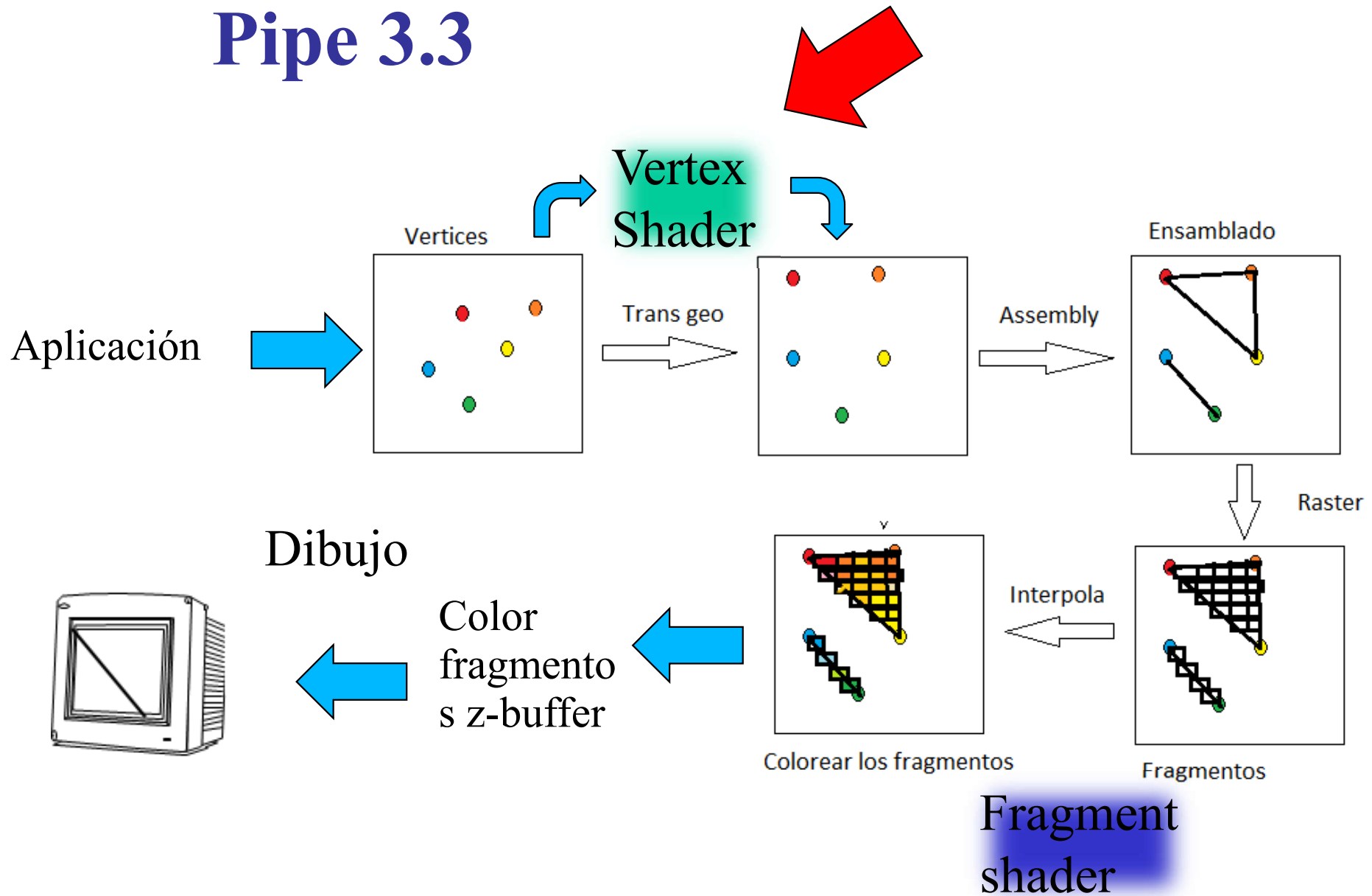
Temario: Tema 4: Transformaciones geométricas, 2D, 3D.

- Tema 4: Transformaciones geométricas, 2D, 3D.
 - 4.1 Traslación, escalado, rotación. (Matrices)
 - 4.2 Coordenadas homogéneas.
 - 4.3 Composición de transformaciones.
 - 4.4 Modelo de visualización.
 - 4.5 Representaciones jerárquicas.

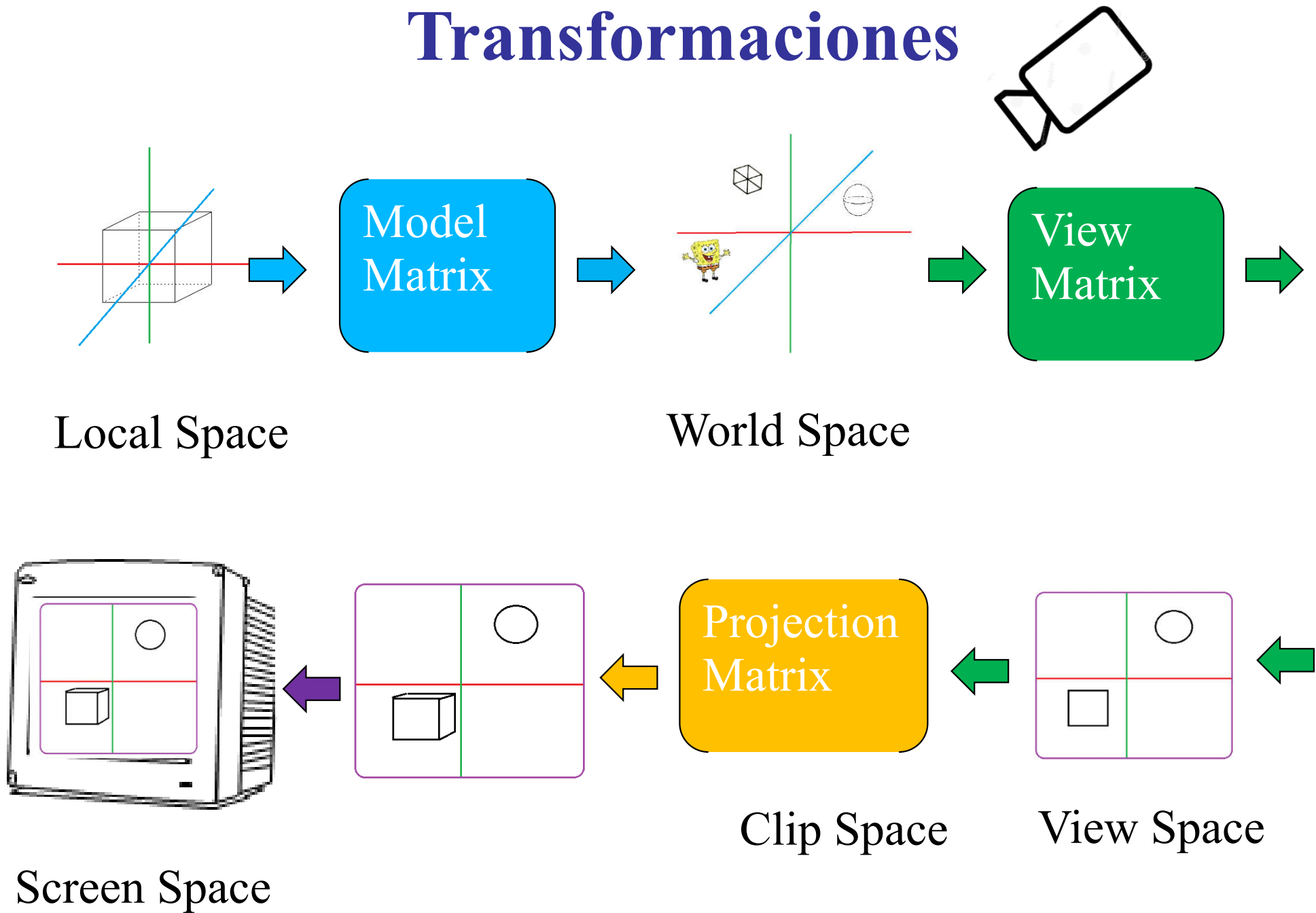
Bibliografía

- GRÁFICOS POR COMPUTADORA CON OPENGL, Hearn, Donald ; Baker, Pauline.
 - Capítulo 5.
- Redbook 8.0 o versión superior.
- Learnopengl.com

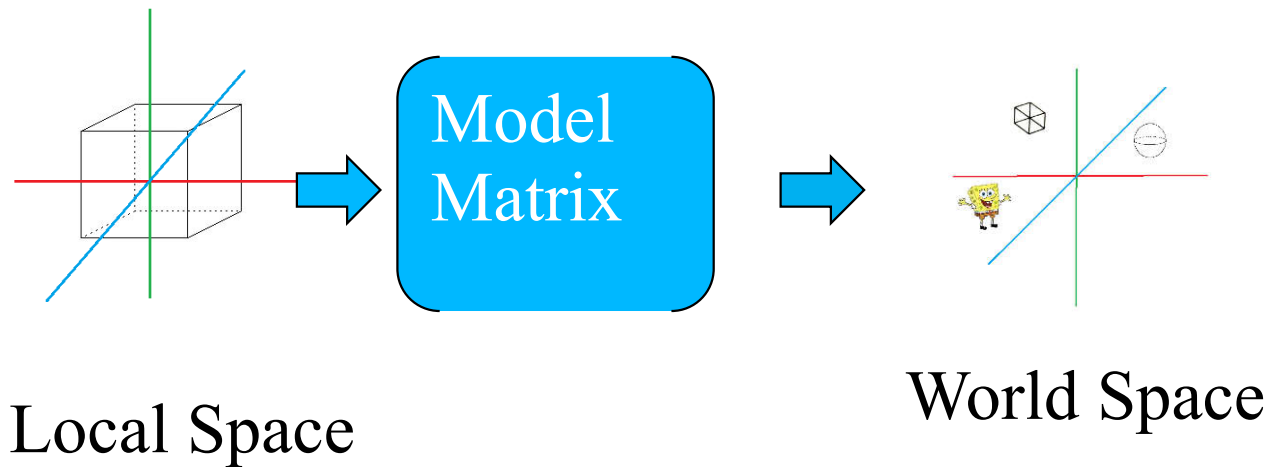
Pipe 3.3



Transformaciones



Transformaciones

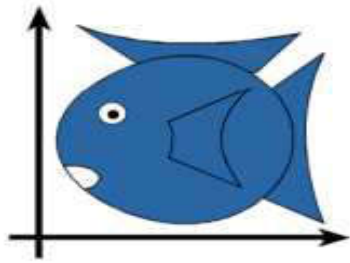


Local Space : Las coordenadas locales son las de un **objeto en relación con su origen local**. Los vértices del objeto están referenciados a estas coordenadas.

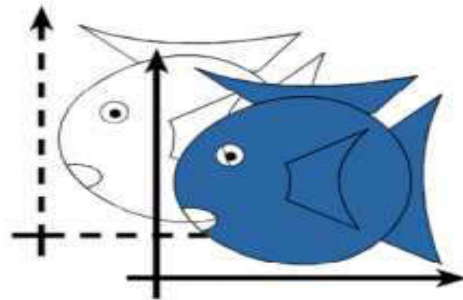
La **matriz del modelo** se encarga de transformar las coordenadas **locales en coordenadas de la escena** (también conocidas como coordenadas de mundo). Comunes a todos los objetos.

World Space: En este espacio, todos **los objetos de la escena** están representados en relación con un **origen global**. La matriz del modelo traslada, escala y rota los objetos, colocándolos en su posición correcta dentro del mundo. Como resultado, las coordenadas de los vértices adquieren nuevos valores en este espacio.

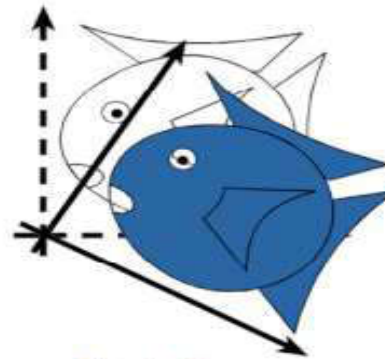
Transformaciones Geométricas



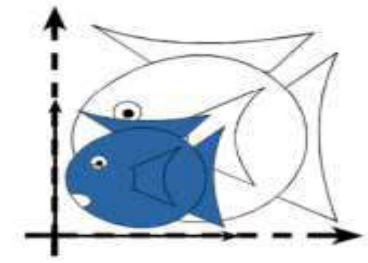
Identity



Translation



Rotation



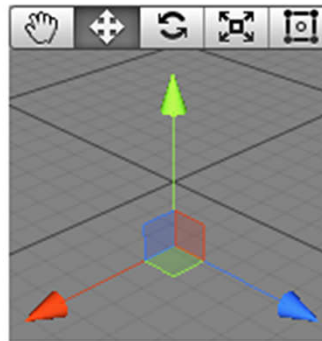
Isotropic
(Uniform)
Scaling

- Las transformaciones se usan para :

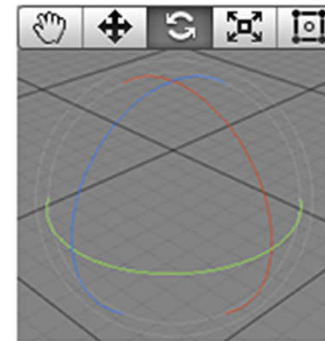
- Trasladar
- Escalar.
- Rotar.

- Permiten

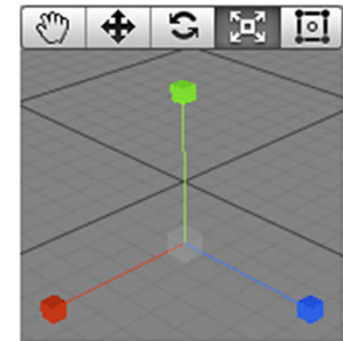
- Posicionar, orientar y variar el tamaño.
- Los objetos y la cámara de la escena.



Translate (W)



Rotate (E)



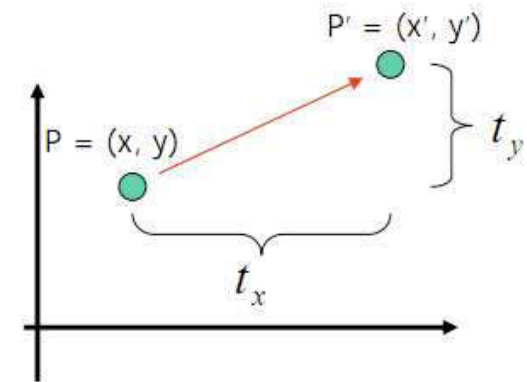
Scale (R)

Traslación

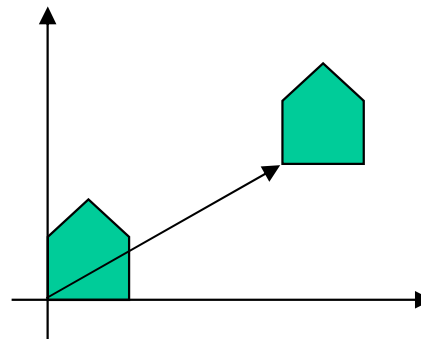
- Llamamos **traslación** a mover un objeto un lugar a otro.
- Para trasladar un punto $P(x, y)$ **sumamos** a sus coordenadas (x, y) las cantidades que lo queremos trasladar, T_x y T_y (vector de desplazamiento
-) obtenemos así $P'(x', y')$ donde: – $P' = P + T$

$$- x' = x + T_x$$

$$- y' = y + T_y \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$



- Si queremos trasladar un objeto trasladaremos cada vértice del objeto



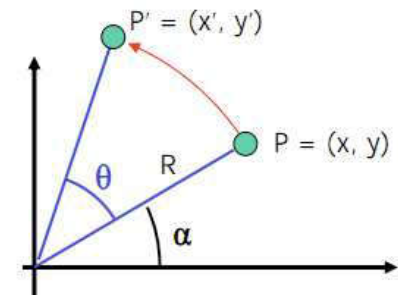
Rotación con respecto al origen

- Llamamos rotación a desplazar un ángulo θ las coordenadas de un punto respecto al origen.
- Rotamos una línea u objeto rotando cada uno de sus vértices
 - $x = r \cos(\alpha)$;
 - $y = r \sin(\alpha)$;
 - $x' = r \cos(\alpha + \theta)$;
 - $y' = r \sin(\alpha + \theta)$;

$$x' = x \cdot \cos(\alpha) - y \cdot \sin(\theta);$$

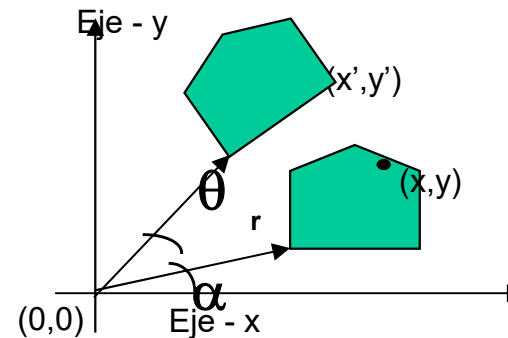
$$y' = x \cdot \sin(\alpha) + y \cdot \cos(\theta);$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



Para calcular las nuevas coordenadas de un objeto bastará **multiplicar**. Todos sus vértices.

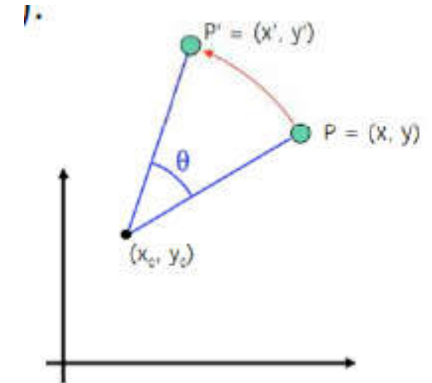
$$\gg P' = R.P$$



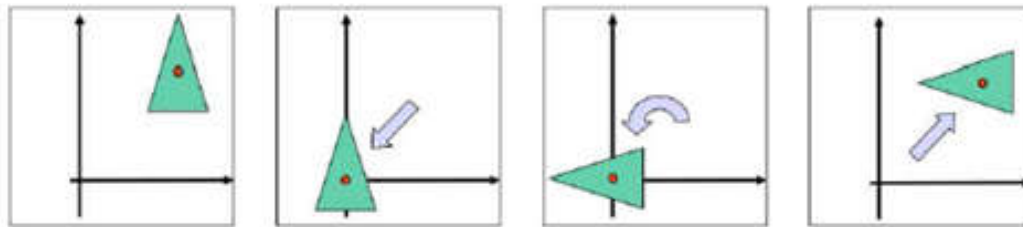
Rotación general

- Fórmula general cuando el punto sobre el cual se rota **no es el origen**, sino un punto cualquiera (x_c , y_c).

$$\begin{cases} x' = x_c + (x - x_c) \cdot \cos \theta - (y - y_c) \cdot \sin \theta \\ y' = y_c + (x - x_c) \cdot \sin \theta + (y - y_c) \cdot \cos \theta \end{cases}$$

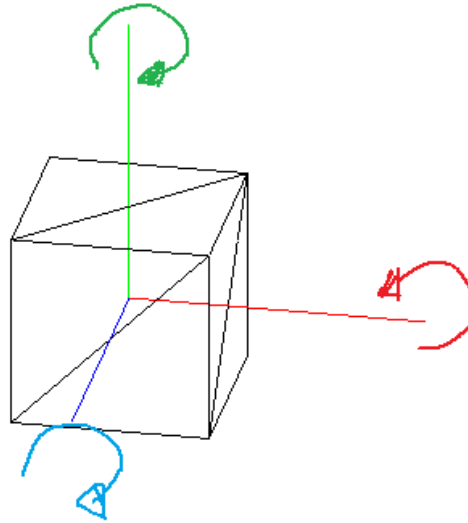


- Como puede verse los pasos son: Translación al origen
 - Rotación alrededor del origen
 - Translación inicial.



$$P' = T(x, y) \cdot R(\alpha) \cdot T(-x, -y) \cdot P(x, y)$$

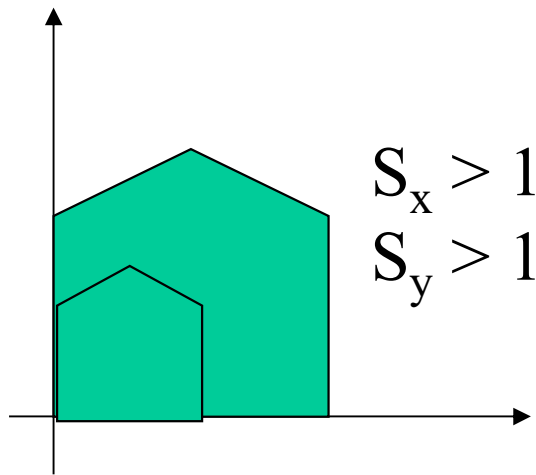
Ángulos de Euler.



- Dados dos sistemas de coordenadas x, y, z y X, Y, Z con un mismo origen, es posible determinar la posición de un sistema en términos del otro usando tres ángulos α, β y γ (yaw, pitch and roll). En nuestro caso serán los ejes de un objeto respecto a un sistema de referencia general.
- Los giros son acumulativos y los ángulos se encuentran ya girados.

Escalado respecto al origen

- Llamamos escalado a cambiar el **tamaño de un objeto**. *No es transformación cuerpo rígido ya que la geometría varia.*
- Para escalar multiplicamos por los coeficientes de escalado que son la razón entre las coordenadas nuevas y las viejas. Escalado respecto al origen.
 - $S_x = \text{Anchura Nueva} / \text{Anchura vieja}$ $0 < S_x < 1$
 - $S_y = \text{Altura Nueva} / \text{Altura vieja}$ $0 < S_y < 1$
 - $S_x = S_y$ Escalado uniforme, $S_x \neq S_y$. Escalado no-uniforme.
- Si queremos escalar un objeto escalamos cada vértice **multiplicamos**.



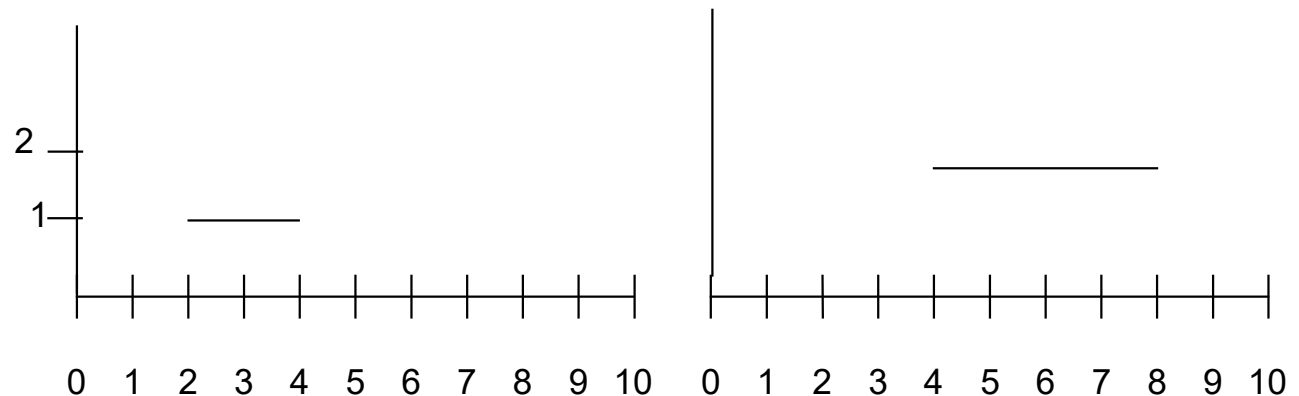
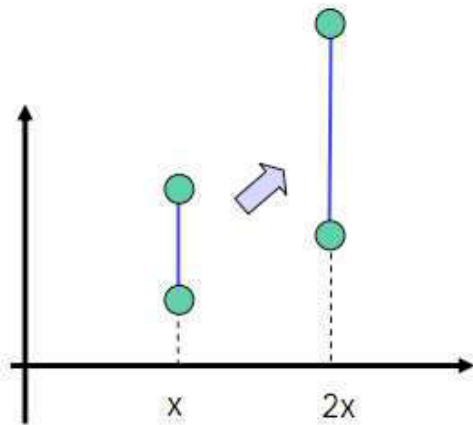
$$\begin{aligned}x' &= S_x \cdot x \\ y' &= S_y \cdot y;\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x \cdot x \\ S_y \cdot y \end{bmatrix}$$

Escalado fuera del origen. Composición

- Si el origen de coordenadas no se encuentra en el interior del objeto, se produce un desplazamiento. El escalado de un objeto fuera del origen implica variar su posición.



- Para evitarlo debemos sustituir el simple escalado por una composición de transformaciones
 - Traslado al origen
 - Escalado al doble
 - Traslado al punto de partida

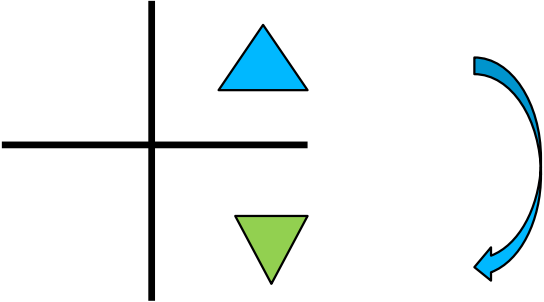
$$\begin{cases} x' = x_c + S_x \cdot (x - x_c) \\ y' = y_c + S_y \cdot (y - y_c) \end{cases}$$



$$P' = T(x, y) \cdot S(S_x, S_y) \cdot T(-x, -y) \cdot P(x, y)$$

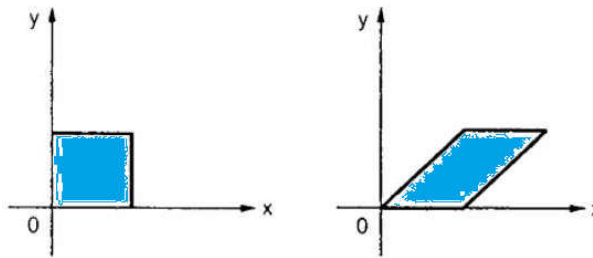
Otras transformaciones

- Denominaremos otras transformaciones a aquellas que hacen uso de la matriz (2D). Reflexión y Shear

$$\begin{bmatrix} r_{11} & r_{12} \\ r_{21} & r_{22} \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$


Shear

$$\begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix}$$



$$\begin{bmatrix} 1 & a \\ b & -1 \end{bmatrix}$$

Se hace share en las dos direcciones.

Reflexión.



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

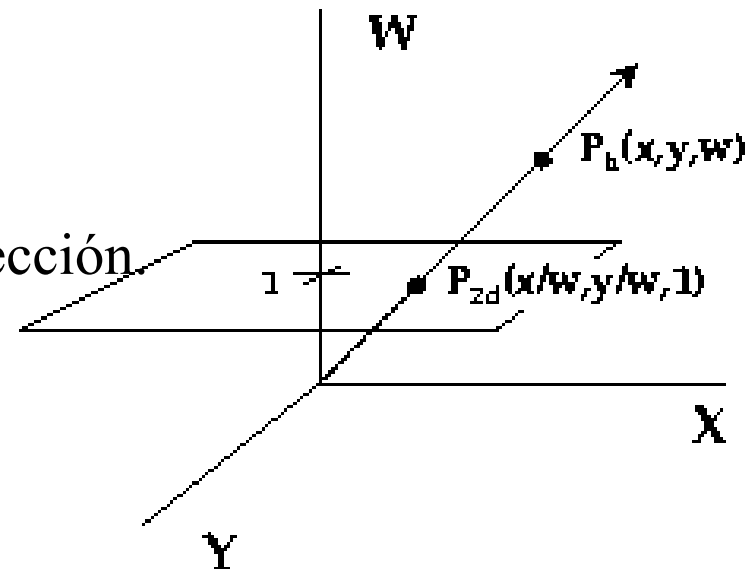
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Coordenadas Homogéneas.

Nos queda por resolver el problema de las traslaciones, que actualmente se implementan mediante operaciones de suma. Para poder expresar **todas las transformaciones geométricas** (traslaciones, rotaciones y escalados) exclusivamente mediante **multiplicaciones de matrices**, y así unificarlas en un sistema coherente, introducimos las **coordenadas homogéneas**. Este sistema de coordenadas ampliado nos permitirá representar cualquier transformación afín como una multiplicación matricial, facilitando la composición y optimización de transformaciones.

Punto $P(x, y, z, w)$

- Vector $P(x, y, z, 0)$ recordar es una dirección.
- Coordenadas homogéneas $w = 1$
- En general $w = 1$
- No existe $(0, 0, 0), (0, 0, 0, 1)$.
- Punto $(x, y, z, 1)$, vector $(x, y, z, 0)$



Transformaciones 2D

$$T(x, y) = T(x, y, 1)$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix} \Rightarrow \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{pmatrix} S_x & 0 & c \\ 0 & S_y & c \\ 0 & 0 & 1 \end{pmatrix}$$

Transformaciones 3D

- Traslación

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Escalado

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Notación en columna M[16].

Por ejemplo, los valores de translación
estarán en M(13,14,15,16)

- Rotación alrededor del eje Z

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Rotación alrededor del eje X

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Rotación alrededor del eje Y

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Orden de aplicación

Caso A: Rotar y luego Escalar

Primero giramos el punto y luego lo escalamos punto (1,0).

1.Rotación: El punto (1, 0) pasa a ser (0, 1).

2.Escalado: Aplicamos escalado al resultado (0, 1). Como el escalado es en X, y nuestra X ahora es 0, el punto no cambia de posición.

1. Cálculo de la Matriz Resultante (M_{SR}):

$$M_{SR} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} (2 \cdot 0 + 0 \cdot 1) & (2 \cdot -1 + 0 \cdot 0) \\ (0 \cdot 0 + 1 \cdot 1) & (0 \cdot -1 + 1 \cdot 0) \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix}$$

2. Multiplicación por el punto P :

$$P' = M_{SR} \cdot P = \begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} (0 \cdot 1 + -2 \cdot 0) \\ (1 \cdot 1 + 0 \cdot 0) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Resultado es $P(1,0)$: **(0, 1)**

Caso B: Escalar y luego Rotar

Primero estiramos el punto y luego lo giramos.

1.Escalado: El punto (1, 0) pasa a ser (2, 0).

2.Rotación: Aplicamos R al resultado (2, 0). El punto gira 90° sobre el origen.

1. Matriz resultante:

$$M_{RS} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 2 & 0 \end{pmatrix}$$

2. Aplicación al punto:

$$P'' = \begin{pmatrix} 0 & -1 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

Resultado para $P(1,0)$: **(0, 2)**

Zonas de una matriz

`glMatrixMode(GL_MODELVIEW);`

- 1.- Transformaciones lineales
- 2.- Traslaciones.
- 3.- Ceros

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

A 4x4 matrix is shown with its elements grouped into four distinct regions, each highlighted with a colored outline:

- Red outline:** The entire 4x4 matrix.
- Blue outline:** The top-left 3x3 sub-matrix, containing elements N_{11} through N_{33} .
- Green outline:** The bottom-right 1x1 element, N_{44} .
- Pink outline:** The bottom row, containing elements 0 , 0 , 0 , and 1 .

$$\begin{bmatrix} N_{11} & N_{21} & N_{31} & N_{41} \\ N_{12} & N_{22} & N_{32} & N_{42} \\ N_{13} & N_{23} & N_{33} & N_{43} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Vamos a acoger el punto (0,0,1) y rotarlo 90 grados en z y trasladarlo 1 unidad en X.

$$\sin(\pi/2.0) = 1; \cos(\pi/2.0) = 0;$$

$$\begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

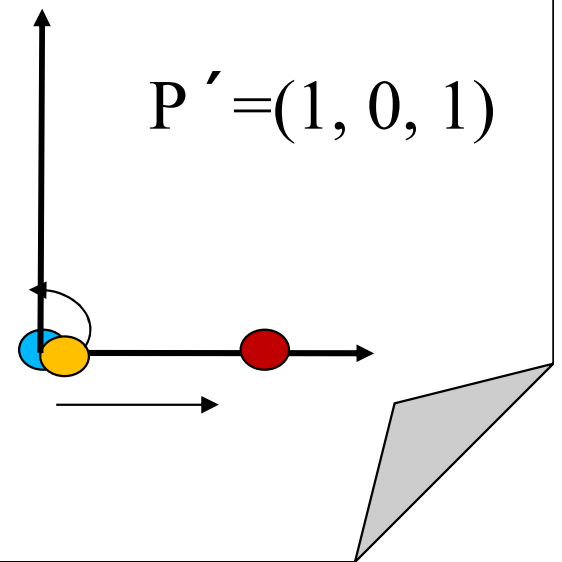
Traslación

Rotación

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

$P' = (1, 0, 1)$



Vamos a acoger el punto (0,0,1) trasladarlo 1 en x y rotarlo 90 grados en z unidad en X.
 $\sin(\pi/2.0) = 1$; $\cos(\pi/2.0) = 0$;

$$\begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix}$$

Traslación

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotación

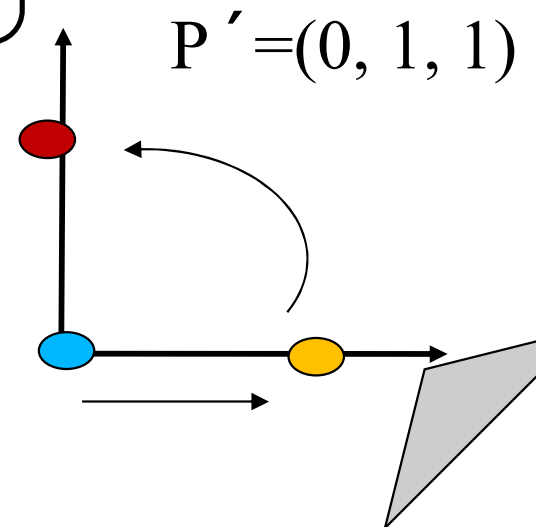
$$\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotación

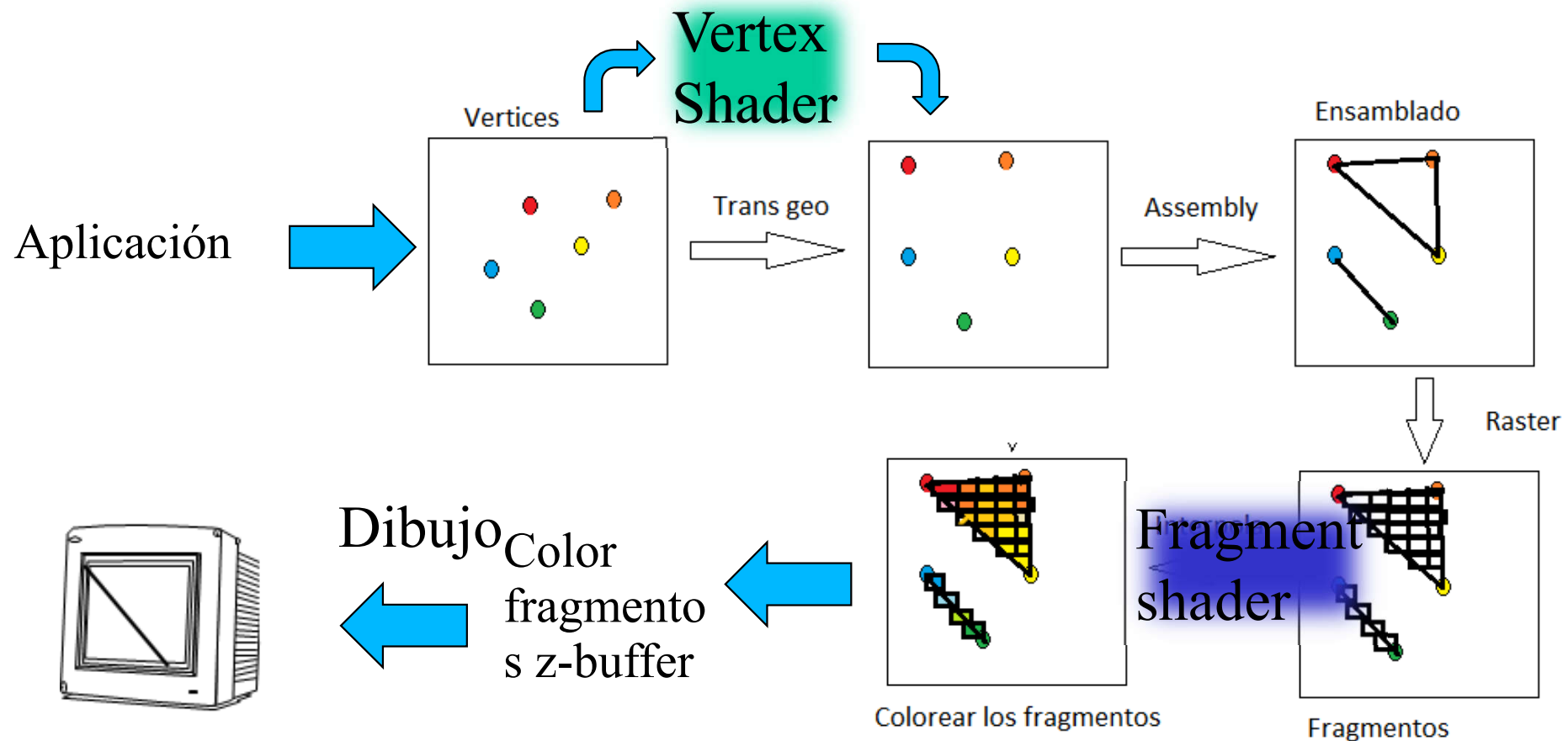
Traslación

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} X' \\ Y' \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$



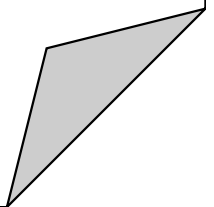
Transformaciones en OpenGL 3.3



El **vertex shader** recibe el vértice y sus atributos (como coordenadas, normales, colores, etc.) y los procesa. Su función principal es calcular la posición final del vértice en el espacio de pantalla (screen space) y, opcionalmente, modificar o ajustar los atributos asociados al vértice, como normales, coordenadas de textura o colores.

Tipo de Shaders

- **Vertex shaders.**
 - Función que recibe como parámetro vértices.
 - Manipula vértices en 3D; Color, coordenadas de textura, orientación y tamaño de vértices.
 - Su propósito es transformar las coordenadas 3D de los vértices en otras coordenadas que ya veremos cuales son.
 - La salida debe ser la posición de los vértices.
- **Fragment shaders (OpenGL)**
 - Recibe la información de cada fragmento.
 - Un fragmento es toda la información necesaria para dibujar un pixel.
 - Se aplica en la última etapa de rasterizado.
 - Su propósito es obtener el color de cada pixel.
 - La salida es el color de cada pixel.
- **Que pasa después del Fragment Shader**
 - Se realizan una serie de comprobaciones relativas al Alpha test, blending test y Depth test.



Como creamos las matrices

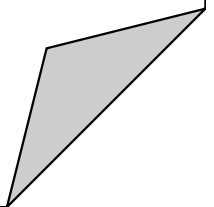
GLM (OpenGL Mathematics) y es una biblioteca .h con lo que no debemos incluir .lib que realiza procesamiento matemático (de esta biblioteca sólo emplearemos la parte de matrices y vectores).

```
#include <glm/glm.hpp>
#include<glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
```

```
glm::mat4 transform; // es la definición de MI matriz de transformacion
transform = glm::mat4(1.0f); // carga identidad
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));
transform = glm::rotate(transform, angulo, glm::vec3(0.0f, 0.0f, 1.0f));
```

- Radians: glm::rotate(matrix, 1.57f, glm::vec3(0, 1, 0)) (rotates 1.57radians)
- Degrees: glm::rotate(matrix, glm::radians(90.0f), glm::vec3(0, 1, 0))

```
transform = glm::scale(transform, glm::vec3(.1, .1, .1));
```



Comunicación CPU → GPU: Variables Uniforms

¿Qué es un Uniform?

Es una variable global que permanece constante para todos los vértices procesados durante una misma llamada de dibujo (*draw call*).

Uso típico: Matrices de transformación (rotación/traslación), color del objeto, tiempo (`u_time`) o parámetros de iluminación.

Flujo de Trabajo en 3 Pasos:

En el Vertex Shader (GLSL):

Se declara fuera del void `main()`.

Ejemplo: `uniform mat4 modelo;`

En la CPU (Localización):

Se solicita a la GPU la dirección de memoria de esa variable.

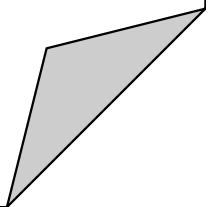
Código: `int loc = glGetUniformLocation(programaID, "modelo");`

En la CPU (Envío de datos):

- Se "inyecta" el valor antes de dibujar.
- *Código:* `glUniform1f(loc, modelo);`

Diferencia Clave:

- Attribute: Un valor distinto por cada vértice (ej. su posición).
- Uniform: Un único valor para todo el objeto (ej. El movimiento)



Vertex Shader

```
#version 330 core
layout (location = 0) in vec3 aPos;
void main()
{
    gl_Position = vec4(aPos, 1.0);
}
```

Model Matrix
(4x4)

```
#version 330 core
layout (location = 0) in vec3 aPos;
uniform mat4 transform;
void main() {
    gl_Position = transform * vec4(aPos, 1.0f);
}
```

Lo enviamos
desde nuestro
programa

Completo

```
glm::mat4 transform; // es la matriz de transformacion
transform = glm::mat4(); // carga identidad
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));
transform = glm::rotate(transform, angulo, glm::vec3(0.0f, 0.0f, 1.0f));
transform = glm::scale(transform, glm::vec3(.1, .1, .1));
```

```
unsigned int transformLoc = glGetUniformLocation(shaderProgram, "modelo");
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
```

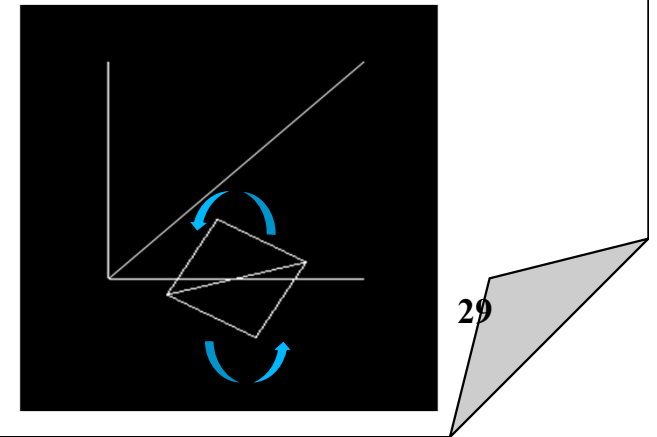
```
glBindVertexArray(VAOCuadrado); // Lo dibujo
glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
```

```
#version 330 core
layout (location = 0) in vec3 aPos;
uniform mat4 modelo;
void main() {
    gl_Position = modelo * vec4(aPos, 1.0f);
}
```

Ejemplo giro. Objeto en un punto sobre si mismo.

```
// Ejes - viewport izquierdo
transform = glm::mat4(1.0f); // matriz identidad
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
glBindVertexArray(VAO);
glDrawArrays(GL_LINES, 0, 8); // 8 vértices = 4 líneas × 2 vértices por línea

// Cuadrado - viewport izquierdo (escala → traslación → rotación)
transform = glm::mat4(1.0f);
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));
transform = glm::rotate(transform, (float)glfwGetTime(), glm::vec3(0.0f, 0.0f, 1.0f));
transform = glm::scale(transform, glm::vec3(0.1f, 0.1f, 0.1f));
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
glBindVertexArray(VAOCuadrado);
glDrawArrays(GL_TRIANGLES, 0, 6);
```



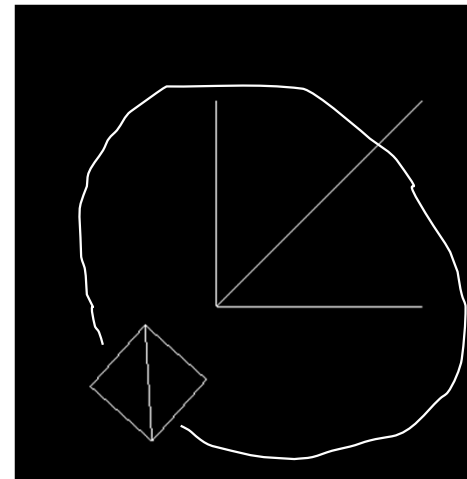
Ejemplo giro. Objeto alrededor del origen

// Ejes - viewport derecho

```
transform = glm::mat4(1.0f);  
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));  
glBindVertexArray(VAO);  
glDrawArrays(GL_LINES, 0, 8);
```

// Cuadrado - viewport derecho (rotación → traslación → escala)

```
transform = glm::mat4(1.0f);  
transform = glm::rotate(transform, (float)glfwGetTime(), glm::vec3(0.0f,  
0.0f, 1.0f));  
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));  
transform = glm::scale(transform, glm::vec3(0.1f, 0.1f, 0.1f));  
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));  
glBindVertexArray(VAOCuadrado);  
glDrawArrays(GL_TRIANGLES, 0, 6);
```



Giro combinado

```
// Ejes - viewport derecho
```

```
transform = glm::mat4(1.0f);
```

```
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
```

```
glBindVertexArray(VAO);
```

```
glDrawArrays(GL_LINES, 0, 8);
```

```
// Cuadrado - viewport derecho (rotación → traslación → escala)
```

```
transform = glm::mat4(1.0f);
```

```
transform = glm::rotate(transform, (float)glfwGetTime(), glm::vec3(0.0f, 0.0f, 1.0f));
```

```
transform = glm::translate(transform, glm::vec3(0.25f, 0.0f, 0.0f));
```

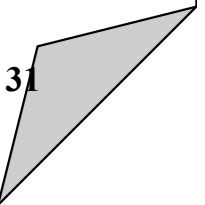
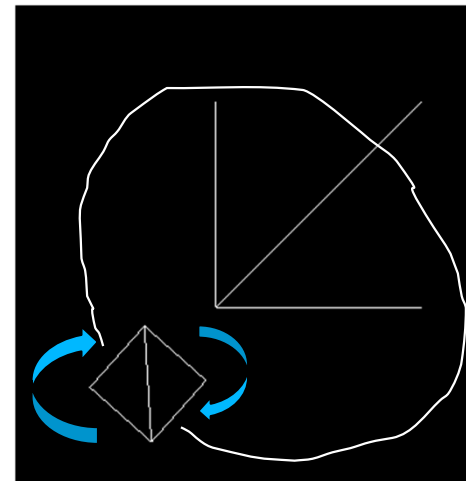
```
transform = glm::rotate(transform, (float)glfwGetTime(), glm::vec3(0.0f, 0.0f, 1.0f));
```

```
transform = glm::scale(transform, glm::vec3(0.1f, 0.1f, 0.1f));
```

```
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
```

```
glBindVertexArray(VAOCuadrado);
```

```
glDrawArrays(GL_TRIANGLES, 0, 6);
```



Movimiento concatenados-jerarquías.

Son movimientos que dependen o posiciones que depende unos de otros, y pueden ser heredados consecutivamente.



Posición Sol	=	P_{sol}				
Posición Mercurio	=	P_{sol}	+	$P_{mercurio}$		
Posición Tierra	=	P_{sol}	+	P_{tierra}		
Posición Luna	=	P_{sol}	+	P_{tierra}	+	P_{luna}
Posición Marte	=	P_{sol}	+	M_{rte}		

`glPushMatrix` pushes the current matrix stack down by one, duplicating the current matrix.

`glPopMatrix` pops the current matrix stack, replacing the current matrix with the one below it on the stack.

Giro Sol

```
typedef struct {  
    char* name;  
    float distancia;  
    float velocidad_trans;  
    float angulo_trans;  
    float velocidad_rot;  
    float angulo_rot;  
    float tamano;  
    int VAO;
```

```
} objetoCeleste;
```

```
objetoCeleste sol = { "Sol", 0, 0, 0, 1, 0, 100, 0,0 };  
objetoCeleste mercurio = {"Mercurio", 200, 2, 0, 0, 0, 5, 0, 0 };  
objetoCeleste tierra = { "tierra",500, 12, 0, 10, 0, 80, 0, 0 };
```

```
//Sol
```

```
//Calculo la Matriz
```

```
transform = glm::mat4(1.0); // identity matrix
```

```
transform = glm::rotate(transform, (float)(sol.angulo_rot), glm::vec3(0.0f, 1.0f, 0.0f));
```

```
transform = glm::scale(transform, glm::vec3(sol.tamano, sol.tamano, sol.tamano));
```

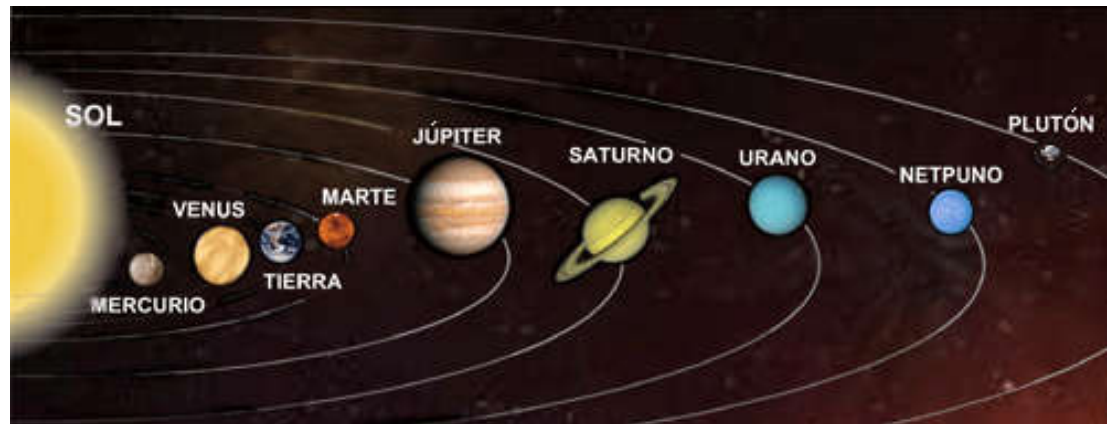
```
//La cargo
```

```
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
```

```
glUniform3fv(colorLoc, 1, glm::value_ptr(glm::vec3(1, 1, 0)));
```

```
glBindVertexArray(sol.VAO);
```

```
glDrawArrays(GL_TRIANGLES, 0, 1080);
```



Giro Tierra-Luna

```
//la tierra //Calculo la Matriz
transform = glm::mat4(1.0); // identity matrix
transform = glm::translate(transform,
glm::vec3(tierra.distancia*cos(tierra.angulo_trans*GradosToRadianes), 0.0,
          tierra.distancia*sin(tierra.angulo_trans*GradosToRadianes)));
transform = glm::rotate(transform, (float)(tierra.angulo_rot), glm::vec3(0.0f, 1.0f, 0.0f));
transform = glm::scale(transform, glm::vec3(tierra.tamano, tierra.tamano, tierra.tamano));

//La carga
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
glUniform3fv(colorLoc, 1, glm::value_ptr(glm::vec3(0, 0, 1)));
glBindVertexArray(tierra.VAO);
glDrawArrays(GL_TRIANGLES, 0, 1080);

////luna Calculo la Matriz
transform = glm::mat4(1.0); // identity matrix
// Lo giro en funcion del tiempo
transform = glm::translate(transform,
glm::vec3(tierra.distancia*cos(tierra.angulo_trans*GradosToRadianes), 0.0,
          tierra.distancia*sin(tierra.angulo_trans*GradosToRadianes)));

//// los de la Luna
transform = glm::translate(transform,
glm::vec3(luna.distancia*cos(luna.angulo_trans*GradosToRadianes), 0.0,
          luna.distancia*sin(luna.angulo_trans*GradosToRadianes)));
transform = glm::rotate(transform, (float)(luna.angulo_rot), glm::vec3(0.0f, 1.0f, 0.0f));
transform = glm::scale(transform, glm::vec3(luna.tamano, luna.tamano, luna.tamano));
//La carga
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));

//ladibujo
glUniform3fv(colorLoc, 1, glm::value_ptr(glm::vec3(.7, .7, .7)));
glBindVertexArray(luna.VAO);
glDrawArrays(GL_TRIANGLES, 0, 1080);
```

```
typedef struct {
float distancia;
float velocidad_trans;
float angulo_trans;
float velocidad_rot;
float angulo_rot;
float tamano;
int listarender;
} objetoCeleste;
```

Movimiento

```
void tiempo() {
    static float unSegundo = 0;
    currentTime = glfwGetTime();
    nbFrames++;
    lapsoTime = currentTime - lastTime;
    unSegundo = unSegundo + lapsoTime;
    if (unSegundo >= 1.0) { // If last printf() was more than 1 sec ago// printf and reset timer
        printf("%f frame/s\n", double(nbFrames));
        nbFrames = 0;
        unSegundo = 0;
    }
    lastTime = currentTime;
}
```

```
typedef struct {
    float distancia;
    float velocidad_trans;
    float angulo_trans;
    float velocidad_rot;
    float angulo_rot;
    float tamano;
    int listarender;
} objetoCeleste;
```

```
void movimiento() {
    sol.angulo_trans += sol.velocidad_trans * lapsoTime;
    if (sol.angulo_trans > 360) sol.angulo_trans -= 360;

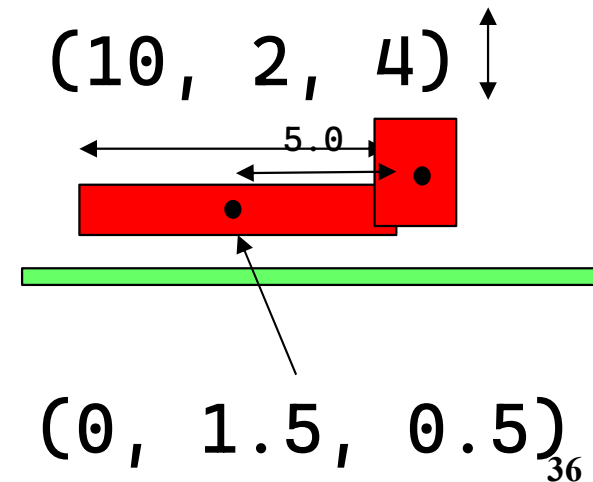
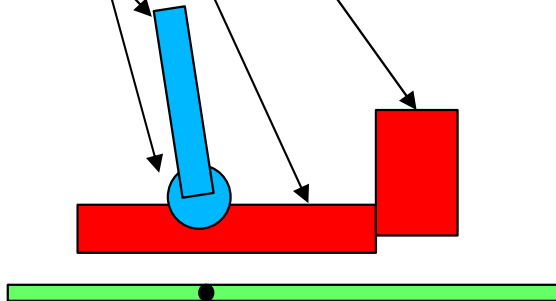
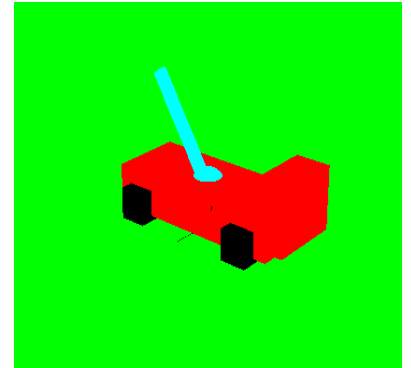
    sol.angulo_rot += sol.velocidad_rot*lapsoTime;
    if (sol.angulo_rot > 360) {
        sol.angulo_rot -= 360;
    }

    tierra.angulo_trans += tierra.velocidad_trans*lapsoTime;
    if (tierra.angulo_trans > 360) tierra.angulo_trans -= 360;

    tierra.angulo_rot += tierra.velocidad_rot*lapsoTime;
    if (tierra.angulo_rot > 360) tierra.angulo_rot -= 360;
}
```

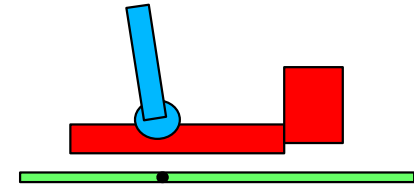
Grúa

```
typedef struct {  
  
    float px, py, pz;    // posicion  
    float angulo_trans;  // angulo giro x  
    float velocidad;     // velocidad de movimiento  
    float sx, sy, sz;    // escalado en los ejes.  
    unsigned int VA0;    // lista de render  
  
} objeto;  
  
objeto base = { 0, 1.5, 0.5, 0, 0, 10, 2, 4, 0 };  
objeto cabina= { 5.0, 1.0, 0, 0, 0, 2.0f, 3.0f, 4.0f, 0};  
objeto Articulacion = { -.4, 0.4 ,0 , 35, 0, 1.0, 1.0, 1.0, 0 };  
objeto Brazo = { 0.0, 3, 0, 0, 0, .5, 6.0, .5, 0 };
```



Grúa

// Coche//Base

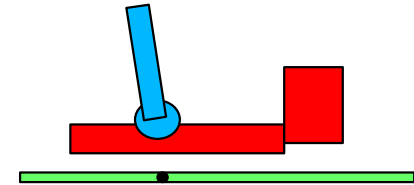


```
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);  
//Lo dibujo
```

```
Mmodel = glm::mat4();  
Mmodel = glm::translate(Mmodel, glm::vec3(base.px, base.py, base.pz));  
Mmodel = glm::rotate(Mmodel, (float)((base.angulo_trans-90) * GradosToRadianes),  
glm::vec3(0.0f, 1.0f, 0.0f));  
TempStack = Mmodel;  
Mmodel = glm::scale(Mmodel, glm::vec3(base.sx, base.sy, base.sz));  
glUniformMatrix4fv(MmodelLoc, 1, GL_FALSE, glm::value_ptr(Mmodel));  
glUniform3fv(colorLoc, 1, glm::value_ptr(glm::vec3(1, 0, 0)));  
glBindVertexArray(base.VAO);  
glDrawArrays(GL_TRIANGLES, 0, 36);
```

```
Mmodel = TempStack;  
Mmodel = glm::translate(Mmodel, glm::vec3(cabina.px, cabina.py, cabina.pz));  
Mmodel = glm::rotate(Mmodel, (float)(cabina.angulo_trans), glm::vec3(1.0f, 0.0f,  
0.0f));  
Mmodel = glm::scale(Mmodel, glm::vec3(cabina.sx, cabina.sy, cabina.sz));  
glUniformMatrix4fv(MmodelLoc, 1, GL_FALSE, glm::value_ptr(Mmodel));  
glBindVertexArray(cabina.VAO);  
glDrawArrays(GL_TRIANGLES, 0, 36);
```

Grúa- calculo posición



```
void tiempo() {
static float unSegundo = 0;
currentTime = glfwGetTime();
nbFrames++;
lapsoTime = currentTime - lastTime;
unSegundo = unSegundo + lapsoTime;
if (unSegundo >= 1.0) { // If last printf() was more than 1 sec ago
// printf and reset timer
printf("%f frame/s\n", double(nbFrames));
nbFrames = 0;
unSegundo = 0;
}
lastTime = currentTime;
}

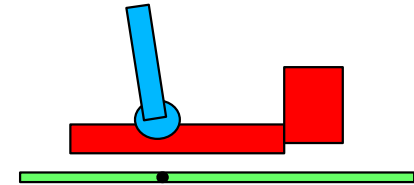
void movimiento() {

base.px = base.px + base.velocidad * sin(base.angulo_trans * GradosToRadianes) *
lapsoTime;
base.pz = base.pz + base.velocidad * cos(base.angulo_trans * GradosToRadianes) *
lapsoTime;

angleWheel += base.velocidad*.1;
if (angleWheel > 360) angleWheel -= 360;

}
```

Grúa- Teclado



```
void processInput(GLFWwindow *window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)    beta += .001;
    if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS)  beta -= .001;
    if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS) alpha += .001;
    if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)  alpha -= .001;

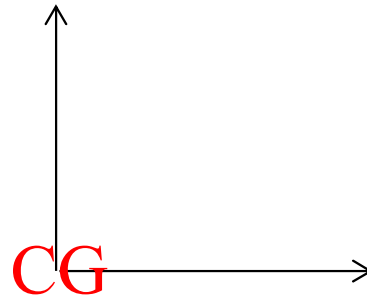
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)    base.velocidad += .01;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)    base.velocidad -= .01;
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)    base.angulo_trans += .5;
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)    base.angulo_trans -= .5;

    if (glfwGetKey(window, GLFW_KEY_L) == GLFW_PRESS)    Articulacion.angulo_trans +=
        .05;
    if (glfwGetKey(window, GLFW_KEY_K) == GLFW_PRESS)    Articulacion.angulo_trans -=
        .05;
}
```

Problema.

1) Representa gráficamente las siguientes transformaciones:

- a) Desplazarme 100 unidades en y , luego rotación de -90 Z.
- b) Rotaciones -90 en Z y 100 unidades en y.



2) Dado el punto (1,1). Calcula matemáticamente.

- a) Lo trasladamos dos unidades en eje Z y lo giramos 45 grados eje X cual es la matriz resultante.
- b) Lo giramos 45 grados en el eje Z y lo trasladamos 2 unidades.

$$\begin{aligned}x' &= x.\cos (\alpha) - y.\text{sen} (\theta); \\y' &= x.\text{sen} (\alpha) + y.\cos (\theta);\end{aligned}$$